

Practice 4: Topological Sorting & Dynamic Dependencies

Deadline: Apr.1/2/3. Must be checked onsite (during labs). Online checks via email or messaging apps are not accepted.

1. Objective

This lab extends the concepts from **Chapter 3.6 (DAGs and Topological Ordering)**. While static topological sorting is standard, real-world systems (like build systems or task schedulers) often involve **dynamic changes** to dependencies.

You will:

1. Implement **Kahn's Algorithm** for static topological sorting (including cycle detection).
2. Implement the **complete Pearce-Kelly dynamic topological sort algorithm** for edge insertion, including:
 - set computation (B/F/I),
 - cycle detection via set intersection,
 - local reordering of affected nodes.

By the end, you will understand how to **maintain a topological order efficiently under dynamic updates**, instead of recomputing from scratch.

2. Background

2.1 Topological Order(Review)

A **topological order** of a directed graph $G = (V, E)$ is an ordering $v_1, v_2, \dots, v_n$ such that for every edge (v_i, v_j) , we have $i < j$.

- Only defined for **DAGs** (Directed Acyclic Graphs).
- If a graph contains a cycle, no valid topological order exists.

2.2 Kahn's Algorithm (Static Topological Sort)

Kahn's algorithm computes a topological order using **in-degrees**:

1. **Preprocessing**: Compute the in-degree (number of incoming edges) of each node in $G = (V, E)$.
2. **Initialization**: Initialize a queue Q containing all nodes with in-degree 0.
3. **Iterative Processing**:
 - Remove node u from Q , append to result *list*.
 - For each edge $u \rightarrow v$:
 - Decrement the in-degree of v by 1
 - If v 's in-degree becomes 0, add v to Q .
4. **Cycle Detection**: After processing, if the length of the result *list* $< |V|$, the graph contains a cycle (and thus no valid topological order).

Key Properties of Kahn's Algorithm

- Time Complexity: $O(|V| + |E|)$ (visits every node and edge exactly once).
- Space Complexity: $O(|V|)$ (for in-degree *array*, *queue*, and *output list*).
- Produces **one valid order** (determined by the queue processing order, not necessarily unique) if and only if the graph is a DAG.

2.3 Pearce-Kelly Algorithm (Dynamic Topological Sort)

Why Dynamic Update?

When a new edge (u, v) is inserted into an existing DAG:

- **Naive approach**: Re-run Kahn's algorithm $\rightarrow O(|V| + |E|)$ every time.
- **Pearce-Kelly**: Only update **affected nodes** $\rightarrow O(|S| + |E(S)|)$, where S is the small affected subset.

Core Idea:

Maintain an array `ord[]` where `ord[x]` = position of node x in the current topological order.

Key Observation:

When inserting edge $u \rightarrow v$:

- └ Case 1: `ord[u] < ord[v]` $\rightarrow$ Order already satisfied! No change needed. ✓
- └ Case 2: `ord[u] > ord[v]` $\rightarrow$ Order violated! May need local adjustment or generate cycle! ⚠

Why Only [ord[v], ord[u]] Range Matters?

Current order: [..., v, ..., u, ...] (ord[v] < ord[u])

New edge: u → v (creates "back-edge")

Nodes with ord[x] < ord[v]:

- └ Either upstream of v, or unrelated to v
- └ After inserting u→v, no new path from u to these nodes exists
- └ Their relative order remains valid ✓

Nodes with ord[x] > ord[u]:

- └ Either downstream of u, or unrelated to u
- └ After inserting u→v, no new path from these nodes to v exists
- └ Their relative order remains valid ✓

Only nodes in [ord[v], ord[u]] may need reordering!

Algorithm Steps (Case 2: ord[u] > ord[v])

Let $R = [ord[v], ord[u]]$ be the affected range.

Step 1: Compute Three Sets within Range R

Set	Definition	How to Compute
B (Backward)	Nodes that can reach u within range R (including u)	BFS/DFS backward from u on reversed graph , filter by $ord[x] \in R$
F (Forward)	Nodes reachable from v within range R (including v)	BFS/DFS forward from v on original graph, filter by $ord[x] \in R$
I (Independent)	Remaining nodes in R: $I = R \setminus (B \cup F)$	Set difference

Step 2: Cycle Detection

If $F \cap B \neq \emptyset \rightarrow$ CYCLE DETECTED! 

Reason: A node $x \in F \cap B$ means:

- x is reachable from v ($v \rightarrow x^*$)
- x can reach u ($x^* \rightarrow u$)
- With new edge $u \rightarrow v$, we get: $v \rightarrow x^* \rightarrow u \rightarrow v$ (cycle!)

Step 3: Local Reordering (If No Cycle)

New order for range R: [B] + [I] + [F]

Where:

- [B]: Nodes in B, preserving their original relative order
- [I]: Nodes in I, preserving their original relative order
- [F]: Nodes in F, preserving their original relative order

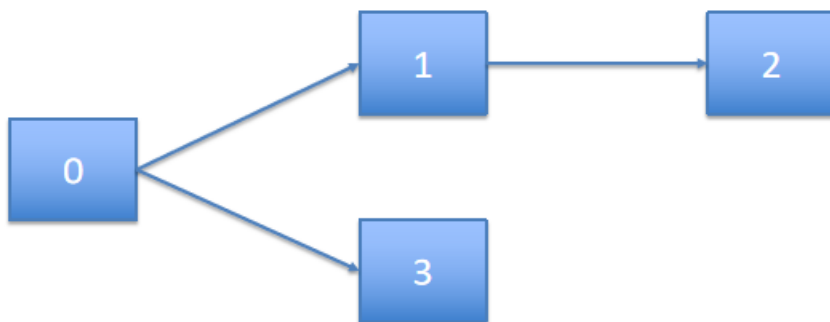
Why this works:

- All edges within B, I, F remain satisfied (original order preserved)
- New edge $u \rightarrow v$: $u^* \in B$, $v^* \in F \rightarrow \text{ord}[u^*] \text{ (in B part)} < \text{ord}[u] < \text{ord}[v] < \text{ord}[v^*] \text{ (in F part)} \checkmark$
- Edges between sets:
 - * $B \rightarrow I$, $B \rightarrow F$, $I \rightarrow F$: original order ensures $\text{ord}[\text{source}] < \text{ord}[\text{target}] \checkmark$
 - * No edges $F \rightarrow B$ or $F \rightarrow I$ or $I \rightarrow B$ can exist (by definition of B,F,I) $\checkmark$

Visual Example Walkthrough

Input: DAG and Topological Order:

Given a DAG and a valid topological order list: 0, 3, 1, 2 .(*position* $\rightarrow$ *node*).



Map the input list to `ord` array, where $\text{ord}[v] = \text{position of node } v$, $\text{node} \rightarrow \text{position}$.

0, 3, 1, 2 $\rightarrow$ $\text{ord}[0]=0$, $\text{ord}[3]=1$, $\text{ord}[1]=2$, $\text{ord}[2]=3 \rightarrow 0, 2, 3, 1$

Original list: *position* $\rightarrow$ *node*

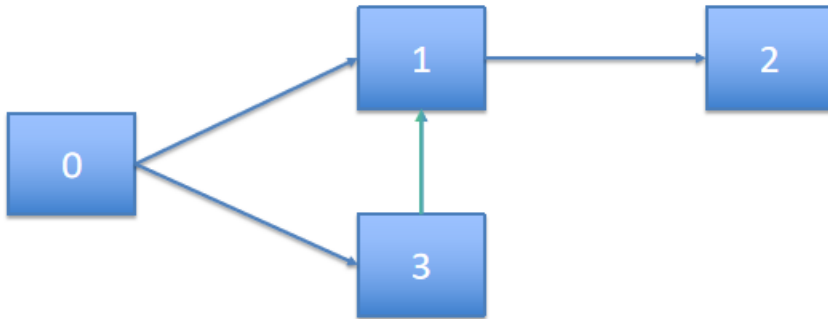
0	1	2	3
0	3	1	2



`ord`: *node* $\rightarrow$ *position*

0	1	2	3
0	2	3	1

Case A: Insert edge $3 \rightarrow 1$



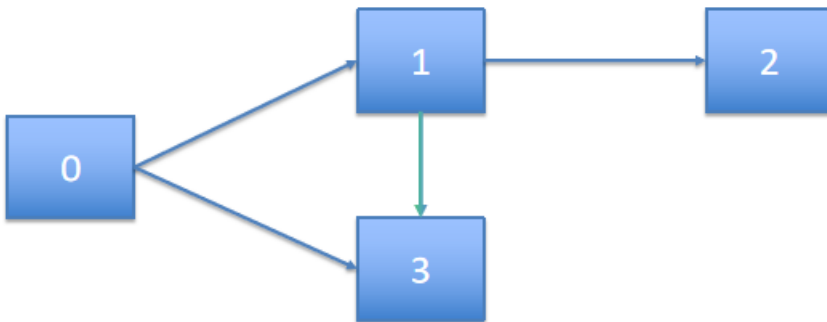
$\text{ord}[3]=1 < \text{ord}[1]=2$

$\text{ord}[3] < \text{ord}[1] \rightarrow$ Already satisfied!

Result: No change needed. ✓

New order: 0, 3, 1, 2

Case B: Insert edge $1 \rightarrow 3$



$\text{ord}[1]=2 > \text{ord}[3]=1$

$\text{ord}[1] > \text{ord}[3] \rightarrow$ May need adjustment!

Range $R = [\text{ord}[3], \text{ord}[1]] = [1, 2] \rightarrow$ Nodes at positions 1,2: $\{3, 1\}$

Step 1: Compute sets within $R=\{3,1\}$:

B (to $u=1$): $\{1\}$ (1 has no incoming edges from nodes in R)

F (from $v=3$): $\{3\}$ (3 has no outgoing edges to nodes in R)

$I = R \setminus (F \cup B) = \emptyset$

Step 2: Cycle check: $B \cap F = \{1\} \cap \{3\} = \emptyset \rightarrow$ No cycle $\checkmark$

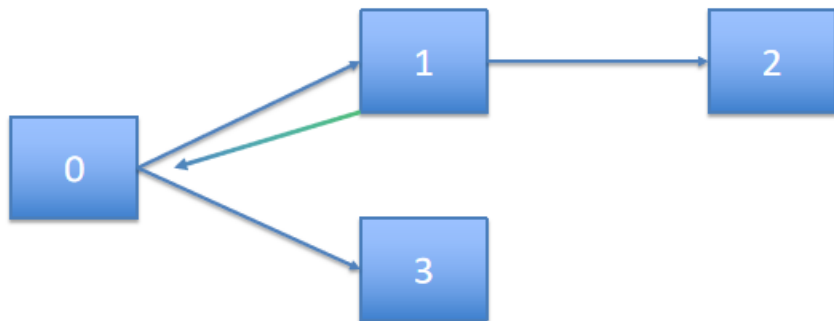
Step 3: Reorder: $[B] + [I] + [F] = [1] + [] + [3] = [1, 3]$

Original order in R was $[3, 1]$, new order is $[1, 3]$

Result: UPDATED

New full order: $0, 1, 3, 2$

Case C: Insert edge $1 \rightarrow 0$ (Creates cycle,)



$\text{ord}[1]=2 > \text{ord}[0]=0$

$\text{ord}[1] > \text{ord}[0] \rightarrow$ May need adjustment!

Range $R = [\text{ord}[0], \text{ord}[1]] = [0, 2] \rightarrow$ Nodes at positions 0,1,2: $\{0, 3, 1\}$

Step 1: Compute sets within $R=\{0,3,1\}$:

B (to $u=1$): $\{0, 1\}$ ($0 \rightarrow 1$ exists, so 0 can reach 1; 1 reaches itself)

F (from $v=0$): $\{0, 1\}$ ($0 \rightarrow 1$ exists, and both in R)

$I = R \setminus (F \cup B) = \{3\}$

Step 2: Cycle check: $B \cap F = \{0,1\} \cap \{0,1\} = \{0,1\} \neq \emptyset \rightarrow$ CYCLE! no

Result: CYCLE (edge $1 \rightarrow 0$ rejected)

Complexity Analysis

Operation	Kahn's (Full)	Pearce-Kelly (Local)
Time	$O(V + E)$	$O(S + E(S))$
When to use	Initial sort, large changes	Small incremental updates

S is affected nodes set

Key Insight: If the new edge only affects a small local region ($|S| \ll V + E$), Pearce-Kelly is dramatically faster.

3. Tasks

Task 1: Static Topological Sort (Kahn's Algorithm) (0.5 points)

Goal: Implement Kahn's Algorithm to compute a valid topological order for a given directed graph, and detect cycles if the graph is not a DAG.

Input Format:

- First line: Two integers n (number of nodes, 0-indexed) and m (number of directed edges).
- Next m lines: Two integers u and v representing a directed edge $u \rightarrow v$.

Output Format:

- If the graph is a DAG: Print the nodes in a valid topological order (space-separated).
- If the graph has a cycle: Print "CYCLE" (no valid topological order exists).

Sample Input 1 (Simple Linear DAG):

```
4 3
0 1
1 2
2 3
```

Expected Output 1:

0 1 2 3

Sample Input 2 (Cyclic Graph):

3 3
0 1
1 2
2 0

Expected Output 2:

CYCLE

Sample Input 3 (Disconnected DAG):

5 3
0 1
2 3
3 4

Expected Output 3 (any valid order):

0 2 1 3 4

Task 2: Dynamic Topological Sort (Pearce-Kelly Algorithm) (0.5 points)

Goal: Implement the complete Pearce-Kelly dynamic topological sort algorithm for edge insertion. You will compute the key sets (B/F/I), detect cycles via set intersection, and reorder the minimal node set to generate a new valid topological order—**no full re-sort of the entire graph is allowed**.

Input Format:

- First line: Two integers n (nodes) and m (edges) for the original DAG.
- Next m lines: Directed edges of the original graph (same format as Task 1).
- Next line: Two integers u and v (the new directed edge $u \rightarrow v$ to insert/test).
- Last line: A space-separated list of n integers (a **valid topological order** of the original DAG).

Output Format:

1. First, print the three key sets in **B F I order** (one set per line, no elements if the set is empty):

B: b1 b2 ...

F: f1 f2 ...

I: i1 i2 ...

2. Then print exactly one result line (per the rules below):

- "CYCLE" : If inserting the edge creates a cycle ($B \cap F \neq \emptyset$).
- "UPDATED" : Followed by the new valid topological order (space-separated) if the edge is safe ($B \cap F = \emptyset$).

Key Requirements:

- You can use BFS/DFS to compute the forward/backward sets (B/F).
- Cycle detection **must** use the set intersection rule ($B \cap F \neq \emptyset$) — no other cycle detection method is allowed.
- Reordering **must** follow the $B \rightarrow I \rightarrow F$ structure (preserve original internal order for each subset).

Test Case 1: Edge Insertion Creates a Cycle ($B \cap F \neq \emptyset$)

Input:

```
4 3
0 1
1 2
2 3
3 0
0 1 2 3
```

Expected Output:

```
B: 0 1 2 3
F: 0 1 2 3
I:
CYCLE
```

Step-by-step:

1. $\text{ord}[3]=3 > \text{ord}[0]=0 \rightarrow$ Need adjustment
2. Position range $R = [0, 3] \rightarrow$ subarray $[0,1,2,3]$
3. B (to $u=3$, within R, on reversed graph): $\{0,1,2,3\}$ (reversed: $3 \rightarrow 2 \rightarrow 1 \rightarrow 0$)

4. F (from $u=0$, within R): $\{0,1,2,3\}$ ($0 \rightarrow 1 \rightarrow 2 \rightarrow 3$)

5. $B \cap F = \{0, 1, 2, 3\} \neq \emptyset \rightarrow \text{CYCLE}$

Test Case 2: Safe Edge Insertion ($B \cap F = \emptyset$, Partial Graph Reorder)

Input:

```
5 4
0 1
1 3
0 2
0 4
3 2
0 1 2 4 3
```

Expected Output:

```
B: 3
F: 2
I: 4
UPDATED 0 1 3 4 2
```

Step-by-step:

1. $\text{ord}[3]=4 > \text{ord}[2]=2 \rightarrow$ Need adjustment
2. Position range $R = [2, 4] \rightarrow$ subarray $[2, 4, 3]$
3. B (to $u=3$, within R , on reversed graph): $\{3\}$
4. F (from $u=2$, within R): $\{2\}$
5. I : $\{4\}$
6. $B+I+F:[3,4,2]$, preserving internal order
7. Update: $0\ 1\ 2\ 4\ 3 \rightarrow 0\ 1\ 3\ 4\ 2$

Test Case 3: Safe Edge Insertion ($B \cap F = \emptyset$, Partial Graph Reorder)

Input:

5 4

$\emptyset$ 1

1 2

3 4

$\emptyset$ 3

4 1

$\emptyset$ 1 2 3 4

Output:

B: 3 4

F: 1 2

I:

UPDATED $\emptyset$ 3 4 1 2

Step-by-step:

1. $\text{ord}[4]=4 > \text{ord}[1]=1 \rightarrow$ Need adjustment
2. Range $R = [1, 4] \rightarrow$ subarray [1, 2, 3, 4]
3. B (to $u=4$, within R, on reversed graph): {3, 4} (reversed: $4 \rightarrow 3$)
4. F (from $v=1$, within R): {1, 2} ($1 \rightarrow 2$)
5. I: empty
6. $B+I+F:[3,4,1,2]$, preserving internal order
7. Update: $\emptyset$ 1 2 3 4 $\rightarrow$ $\emptyset$ 3 4 1 2

Test Case 4: Safe Edge Insertion (No adjustment)

Input:

```
5 4
0 1
1 2
3 4
0 3
2 4
0 1 2 3 4
```

Output:

```
B:
F:
I:
UPDATED 0 1 2 3 4
```

Step-by-step:

1. $\text{ord}[2]=2 < \text{ord}[4]=4 \rightarrow$ No adjustment
2. B:empty
3. F:empty
4. I:empty
5. Output original list: 0 1 2 3 4

Self-Check Questions

1. **Task 1:** Does your code correctly handle disconnected DAGs (multiple nodes with in-degree 0)?
2. **Task 2:** Do you use BFS/DFS to compute the backward set B (on the reversed graph) and forward set F (on the original graph)?
3. **Task 2:** Does your reordering strictly follow the $B \rightarrow I \rightarrow F$ structure (preserving original internal order)?

4. Submission

Show the following to your tutor or SA **onsite**:

1. **Source Code:** Well-commented code for both tasks.
2. **Sample Run Results:** Terminal output/screenshots for **all test cases** in Section 3 (Task 1 + Task 2).

3. **Verbal Explanation:** Explain the following in your own words (no script reading):

- How Kahn's algorithm works (including cycle detection).
- The core rules of the Pearce-Kelly algorithm:
 - a. What B, F, and I represent, and how to compute them.
 - b. Why $B \cap F \neq \emptyset$ implies a cycle when inserting $u \rightarrow v$.
 - c. Why $[B] + [I] + [F]$ reordering guarantees a valid topological order for the new edge.
- Why Pearce-Kelly is more efficient than re-running Kahn's algorithm for dynamic DAG updates.
- Under what conditions does the affected set in Pearce-Kelly satisfy , causing the algorithm to degrade to the same complexity as Kahn's?

Grading:

- Submissions on time: 0.5–1 points.
- Late submissions: 0 points.

Reference

[A Dynamic Topological Sort Algorithm for Directed Acyclic Graphs](#)